

Artifact: Dependency Triad: A Metric to Quantify the Dependencies Between Attributes for Local Differential Privacy

Sandaru Jayawardana

The University of Sydney

Sydney, Australia

sandaru.jayawardana@sydney.edu.au

Ming Ding

Technology, CSIRO

Sydney, Australia

ming.ding@csiro.au

Sennur Ulukus

University of Maryland

College Park, USA

ulukus@umd.edu

Kanchana Thilakarathna

The University of Sydney

Sydney, Australia

kanchana.thilakarathna@sydney.edu.au

A Artifact Appendix

A.1 Abstract

This artifact provides the Python implementation of the *Dependency Triad* (DT) metric. Given a conditional probability matrix and, optionally, an entry-wise distributional uncertainty matrix, it computes the DT parameters (α, β, δ) and uses them to estimate correlation-induced privacy leakage (CPL) for a given privacy budget ϵ . The repository contains source code, synthetic data, and a Jupyter notebook smoke test. Running the notebook produces three plots: DT's CPL estimates, the corresponding estimation error, and DT's CPL estimates under different uncertainty levels. The source code is released under the Apache License 2.0; datasets remain subject to their original licenses.

A.2 Description & Requirements

The repository root contains README.md, LICENSE, test.ipynb, and requirements.txt. src/CPL_metrics/DT.py contains the implementation of Algorithms 1 and 2 of DT, and CPL_metrics/ contains the implementations of LTM, HCC-1, and HCC-2. These competing CPL-analysis algorithms were reimplemented based on their respective original papers. Synthetic inputs are stored under datasets/synthetic/. The root README.md provides the complete usage instructions.

A.2.1 Security, privacy, and ethical concerns. No special security or privacy risks are identified. The artifact runs as a local Python/Jupyter workflow and does not require disabling security mechanisms or performing destructive operations.

A.2.2 How to access. The maintained artifact is available at <https://github.com/SandaruJayawardana/dependency-triad-ccs26>.

A.2.3 Hardware dependencies. None. A standard CPU-based computer capable of running Python 3 and Jupyter is sufficient.

A.2.4 Software dependencies. The artifact requires Python 3 and the packages listed in requirements.txt: NumPy, SciPy, Matplotlib, pandas, and seaborn. Git and Python's venv module are used by the installation procedure.

A.2.5 Benchmarks. The supported benchmark uses the synthetic data in datasets/synthetic/. The smoke test produces the CPL-estimation and estimation-error plots corresponding to Figures 4(a) and 4(b) in the paper, and a third plot showing DT's CPL estimate

at different uncertainty levels (corresponding to Figure 7(b) in the paper). The artifact targets the core DT implementation and does not reproduce every figure or full-scale experiment in the paper.

A.3 Set Up

A.3.1 Installation. Clone the repository and install the dependencies in a virtual environment (the commands below assume a Unix-like shell):

```
git clone REPOSITORY_URL
cd dependency-triad-ccs26
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install jupyter nbconvert ipykernel
pip install -r requirements.txt
```

A.3.2 Basic test. Start Jupyter with jupyter notebook test.ipynb and run all notebook cells. Successful execution produces three plots. The first two show the estimated CPL and its estimation error, matching Figures 4(a) and 4(b); the third shows the CPL estimated by DT under different uncertainty levels, matching Figure 7(b).

A.4 Evaluation Workflow

A.4.1 Major claims.

- (C1): The artifact implements Algorithms 1 and 2 and computes the DT parameters (α, β, δ) from a conditional probability matrix, with optional distributional uncertainty. It can also estimate CPL either from a conditional probability matrix or from previously computed DT parameters. This claim is validated by experiment (E1).
- (C2): The supplied smoke test demonstrates DT's CPL estimation, estimation error, and behavior under different uncertainty levels. Its first two outputs match Figures 4(a) and 4(b) in the paper, and third matches Figure 7(b). This claim is validated by experiment (E1).

A.4.2 Experiments.

- (E1): **DT smoke test** [approximately 10 person-minutes and a few compute-minutes on a standard computer, excluding dependency download time].

Preparation: Clone the repository, create and activate the virtual environment, and install the dependencies as specified above.

Execution: From the repository root, run `jupyter notebook test.ipynb`, open the notebook if necessary, and execute all cells in order.

Results: Confirm that execution completes without errors and produces three plots. The first two report CPL estimates and their estimation error, corresponding to Figures 4(a) and 4(b); the third reports DT’s CPL estimate for multiple uncertainty levels, corresponding to Figure 7(b).

A.5 Notes on Reusability

The main entry point is DT in `src/CPL_metrics/DT.py`:

```
DT(eps, CMF=0, Delta=0, leakage_only=False,
```

```
DT_parameters=(0, 0, 0), get_parameters=False)
```

Users can supply their own privacy budget ϵ , conditional probability matrix CMF representing $P(\hat{X}|X_k)$, and optional uncertainty matrix Delta.

When both `get_parameters=False` and `leakage_only=False`, the function estimates CPL directly from CMF, Delta, and ϵ . Setting `get_parameters=True` returns DT parameters (α, β, δ) ; setting `leakage_only=True` estimates CPL directly from a supplied DT_parameters tuple and privacy budget ϵ . This interface allows DT to be applied to other conditional distributions without modifying the core implementation.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.